

aligned_accessor: An mdspan accessor expressing pointer overalignment

Document #: P2897R0
Date: 2023-05-18
Project: Programming Language C++ LEWG
Reply-to: Mark Hoemmen
<mhoemmen@nvidia.com>
Damien Lebrun-Grandie
<lebrungrandt@ornl.gov>
Nicolas Morales
<nmmoral@sandia.gov>
Christian Trott
<crtrott@sandia.gov>

Contents

- [1 Authors](#)
- [2 Revision history](#)
- [3 Purpose of this paper](#)
- [4 Key features](#)
- [5 Design discussion](#)
 - [5.1 The accessor is not nestable](#)
- [6 Implementation](#)
- [7 Example](#)
- [8 Wording](#)
 - [8.1 Add subsection !\[\]\(38441ceaa711016e0bf2ad46ad394ff4_img.jpg\) \[mdspan.accessor.aligned\] with the following](#)
 - [8.2 Members \[mdspan.accessor.aligned.members\]](#)

§ 1 Authors

- Mark Hoemmen (mhoemmen@nvidia.com) (NVIDIA)
- Damien Lebrun-Grandie (lebrungrandt@ornl.gov) (Oak Ridge National Laboratory)

— Nicolas Morales (nmmoral@sandia.gov) (Sandia National Laboratories)

— Christian Trott (crtrott@sandia.gov) (Sandia National Laboratories)

§ 2 Revision history

— Revision 0 (pre-Varna) to be submitted 2023-05-19

§ 3 Purpose of this paper

We propose adding `aligned_accessor` to the C++ Standard Library. This class template is an `mdspan` accessor policy that uses `assume_aligned` to decorate pointer access. We think it belongs in the Standard Library for two reasons. First, it would serve as a common vocabulary type for interfaces that take `mdspan` to declare their minimum alignment requirements. Second, it extends to `mdspan` accesses the optimizations that compilers can perform to pointers decorated with `assume_aligned`.

`aligned_accessor` is analogous to the various `atomic_accessor_*` templates proposed by P2689. Both that proposal and this one start with a Standard Library feature that operates on a “raw” pointer (`assume_aligned` or the various `atomic_ref*` templates), and then propose an `mdspan` accessor policy that straightforwardly wraps the lower-level feature.

We had originally written `aligned_accessor` as an example in P2642, which proposes “padded” `mdspan` layouts. We realized that `aligned_accessor` was more generally applicable and that standardization would help the padded layouts proposed by P2642 reach their maximum value.

§ 4 Key features

- `offset_policy` is `default_accessor`
- `data_handle_type` is `ElementType*`
- Constructor permits conversion
 - from `nonconst` to `const ElementType`, and
 - from more overalignment to less overalignment
- `is_sufficiently_aligned` checks pointer alignment

The `offset_policy` alias is `default_accessor<ElementType>`, because even if a pointer `p` is aligned, `p + i` might not be.

The `data_handle_type` alias is `ElementType*`. It needs no further adornment, because alignment is asserted at the point of access, namely in the access function. Some

implementations might have an easier time optimizing if they also apply some implementation-specific attribute to `data_handle_type` itself. Examples of such attributes include `__declspec(align_value(byte_alignment))` and `__attribute__((align_value(byte_alignment)))`. However, these attributes should not apply to the result of `offset`, for the same reason that `offset_policy` is `default_accessor` and not `aligned_accessor`.

The constructor is analogous to `default_accessor`'s constructor, in that it exists to permit conversion from `nonconst element_type` to `const element_type`. The constructor of `aligned_accessor` additionally permits conversion from more overalignment to less overalignment – something that we expect users may need to do. For example, users may start with `aligned_accessor<float, 128>`, because their allocation function promises 128-byte alignment. However, they may then need to call a function that takes an `mdspan` with `aligned_accessor<float, 32>`, which declares the function's intent to use 8-wide SIMD of `float`.

The `is_sufficiently_aligned` function checks whether a pointer has sufficient alignment to be used correctly with the class. This makes it easier for users to check preconditions, without needing to know how to cast a pointer to an integer of the correct size and signedness.

§ 5 Design discussion

§ 5.1 The accessor is not nestable

We considered making `aligned_accessor` “wrap” any accessor type meeting the right requirements. For example, `aligned_accessor` could take the inner accessor as a template parameter, store an instance of it, and dispatch to its member functions. That would give users a way to apply multiple accessor “attributes” to their data handle, such as atomic access (see P2689) and overalignment.

We decided against this approach for three reasons. First, we would have no way to validate that the user's accessor type has the correct behavior. We could check that their accessor's `data_handle_type` is a pointer type, but we could not check that their accessor's access function actually dereferences the pointer. For instance, access might instead interpret the data handle as a file handle or a key into a distributed data store.

Second, even if the inner accessor's access function actually did return the result of dereferencing the pointer, the outer access function might not be able to recover the effects of the inner access function, because access computes a reference, not a pointer. For example, our `aligned_accessor`'s access function can only apply `assume_aligned` to the pointer; knowledge of overalignment at compile time “lost” with the dereference at element `i`.

Third, any way (not just this one) of nesting two generic accessors raises the question of order dependence. Even if it were possible to apply the effects of both the inner and outer


```

template<class ElementType, std::size_t byte_alignment>
allocation<ElementType> allocate_raw(const std::size_t num_elements)
{
    const std::size_t num_bytes = num_elements * sizeof(ElementType);
    void* ptr = std::aligned_alloc(byte_alignment, num_bytes);
    return {ptr, delete_raw<ElementType>{}};
}

float user_function(size_t num_elements, float alpha)
{
    // Code using the above two interfaces needs to allocate to the max
    // alignment.
    // Users could also query aligned_accessor::byte_alignment
    // for the various interfaces and take the max.
    constexpr size_t max_byte_alignment = 32;
    auto x_alloc = allocate_raw<float, max_byte_alignment>(num_elements);
    auto y_alloc = allocate_raw<float, max_byte_alignment>(num_elements);

    aligned_mdspan<max_byte_alignment> x(x_alloc.get());
    aligned_mdspan<max_byte_alignment> y(y_alloc.get());

    fill_x(x); // automatic conversion from 32-byte aligned to 16-byte
               // aligned
    fill_y(y); // automatic conversion again

    vectorized_axpy(y, alpha, x);
    return vectorized_norm(y);
}

```

§ 8 Wording

Text in blockquotes is not proposed wording, but rather instructions for generating proposed wording. The  character is used to denote a placeholder section number which the editor shall determine.

In `[version.syn]`, add

```
#define __cpp_lib_aligned_accessor YYYYMMML // also in <mdspan>
```

Adjust the placeholder value as needed so as to denote this proposal's date of adoption.

To the Header `<mdspan>` synopsis **[mdspan.syn]**, after class `default_accessor` and before class `mdspan`, add the following.

```

// [mdspan.accessor.aligned], class template aligned_accessor
template<class ElementType, size_t byte_alignment>
class aligned_accessor;

```

At the end of **[mdspan.accessor.default]** and before **[mdspan.mdspan]**, add all the material that follows.

§ 8.1 Add subsection [mdspan.accessor.aligned] with the following

 Class template `aligned_accessor` [mdspan.accessor.aligned]

 1 Overview [mdspan.accessor.aligned.overview]

```
template<class ElementType, size_t the_byte_alignment>
struct aligned_accessor {
    using offset_policy = default_accessor<ElementType>;
    using element_type = ElementType;
    using reference = ElementType&;
    using data_handle_type = ElementType*;

    static constexpr size_t byte_alignment = the_byte_alignment;

    constexpr aligned_accessor() noexcept = default;

    template<class OtherElementType, size_t other_byte_alignment>
    constexpr aligned_accessor(
        aligned_accessor<OtherElementType, other_byte_alignment>) noexcept;

    constexpr operator default_accessor<element_type>() const {
        return {};
    }

    constexpr reference access(data_handle_type p, size_t i) const noexcept;

    constexpr typename offset_policy::data_handle_type
        offset(data_handle_type p, size_t i) const noexcept;

    constexpr static bool is_sufficiently_aligned(data_handle_type p);
};
```

- 1 *Mandates:* `byte_alignment` is a power of two.
- 2 `aligned_accessor` meets the accessor policy requirements.
- 3 `ElementType` is required to be a complete object type that is neither an abstract class type nor an array type.
- 4 Each specialization of `aligned_accessor` is a trivially copyable type that models semiregular.
- 5 $[0, n)$ is an accessible range for an object `p` of type `data_handle_type` and an object of type `aligned_accessor` if and only if $[p, p + n)$ is a valid range.

§ 8.2 Members [mdspan.accessor.aligned.members]

```
template<class OtherElementType, size_t other_byte_alignment>
constexpr aligned_accessor(
    aligned_accessor<OtherElementType, other_byte_alignment>) noexcept {}
```

1 *Constraints:*

- (1.1) — `is_convertible_v<OtherElementType(*)[], element_type(*)[>` is true, and
- (1.2) — `gcd(other_byte_alignment, byte_alignment) == byte_alignment` is true.

```
constexpr reference access(data_handle_type p, size_t i) const noexcept;
```

- 2 *Preconditions:* `p` points to an object `x` of a type similar (**[conv.qual]**) to `element_type`, where `x` has alignment `byte_alignment` (**[basic.align]**).

- 3 *Effects:* Equivalent to: `return assume_aligned<byte_alignment>(p)[i];`

```
constexpr typename offset_policy::data_handle_type  
offset(data_handle_type p, size_t i) const noexcept;
```

- 4 *Preconditions:* `p` points to an object `x` of a type similar (**[conv.qual]**) to `element_type`, where `x` has alignment `byte_alignment` (**[basic.align]**).

- 5 *Effects:* Equivalent to: `return p + i;`

```
constexpr static bool is_sufficiently_aligned(data_handle_type p);
```

- 6 *Preconditions:* `p` points to an object `x` of a type similar (**[conv.qual]**) to `element_type`.

- 7 *Returns:* true if `x` has alignment at least `byte_alignment`, else false.